

Modern Software Architecture & Engineering Principles

SOLID, DRY, Cloud Computing,
DevOps, Microservices, Docker &
Kubernetes

SOLID Principles

- Set of 5 object-oriented design principles
 - Improve maintainability, scalability, and testability
 - Commonly used in enterprise-level applications

S – Single Responsibility Principle

- A class should have only one reason to change
 - Each class handles one responsibility
 - Example: Invoice class should not handle printing or saving

O – Open/Closed Principle

- Open for extension, closed for modification
 - Add new behavior without changing existing code
 - Example: Add new payment method via interface

L – Liskov Substitution Principle

- Derived classes must be substitutable for base classes
 - Child class should not break parent behavior
 - Example: Square should not break Rectangle logic

I – Interface Segregation Principle

- No client should be forced to implement unused methods
 - Prefer multiple small interfaces
 - Example: Printer vs MultiFunctionPrinter interfaces

D – Dependency Inversion Principle

- Depend on abstractions, not concrete classes
 - High-level modules should not depend on low-level modules
 - Example: Service depends on interface, not implementation

DRY Principle (Don't Repeat Yourself)

- Avoid duplication of code or logic
 - Each piece of knowledge should have a single source
 - Improves maintainability and reduces bugs

DRY – Example

- Validation logic written once in a utility or service
 - Avoid copy-pasting same SQL or business logic
 - Used heavily in microservices and APIs

Cloud Computing – Basics

- On-demand delivery of computing resources over the internet
 - Pay-as-you-go pricing model
 - Highly scalable and flexible

Cloud Service Models

- SaaS – Software as a Service
 - PaaS – Platform as a Service
 - IaaS – Infrastructure as a Service

SaaS, PaaS, IaaS – Examples

- SaaS: Gmail, Salesforce
 - PaaS: Heroku, Google App Engine
 - IaaS: AWS EC2, Azure Virtual Machines

Agile Methodology

- Iterative and incremental development
 - Focus on customer collaboration
 - Quick feedback and adaptability

DevOps Principles

- Collaboration between Development and Operations
 - Automation of build, test, and deployment
 - Continuous monitoring and feedback

CI/CD & Automation

- CI: Continuous Integration – frequent code integration
 - CD: Continuous Deployment/Delivery
 - Tools: Jenkins, GitHub Actions, GitLab CI

Microservices Architecture

- Application broken into small independent services
 - Each service focuses on a business capability
 - Communicate via APIs

Key Microservices Concepts

- Decentralized Data Management
 - Independent Deployment
 - Polyglot Persistence and Programming

Microservices – Explanation

- Each service owns its own database
 - Services can be deployed independently
 - Different languages and databases can be used

Docker & Containerization

- Docker is a containerization platform
 - Containers package app + dependencies
 - Lightweight and fast compared to VMs

Docker Concepts

- Images – Read-only application templates
 - Containers – Running instances of images
 - Dockerfile – Instructions to build image

Docker Hub & Registries

- Docker Hub – Public image repository
 - Private registries for enterprise use
 - Versioned image management

Kubernetes – Overview

- Container orchestration platform
 - Manages containerized applications
 - Originally developed by Google

Kubernetes Features

- Orchestration and Automation
 - Service Discovery and Load Balancing
 - Self-healing mechanisms

Kubernetes – Self-Healing

- Automatically restarts failed containers
 - Reschedules containers on failure
 - Ensures desired state is maintained